B.K. BIRLA COLLEGE, KALYAN
(EMPOWERED AUTONOMOUS STATUS)

(DEPARTMENT OF COMPUTER SCIENCE)

CERTIFICATE

This is to certify that **Mr. Kajol Rabindra Epili**, Roll No. 11, Examination Seat No. 33261014 has satisfactorily completed the Practical in **JAVA programming** of Semester 3. for the fulfillment of **S.Y. B.Sc. (Computer Science) for the year 2026-2027.**

Date : / / 2026

Place : Kalyan

Professor-In-Charge

Head Of Department
(Computer Science)

INDEX

Programme: _____ Semester: _____

Course: _____

Sr. No.	Prac. No.	Particulars	Date	Teacher Signature
1	1	Write a Java program for basic arithmetic operations		
2	2	Program using conditional and looping statements		
3	3	Program demonstrating classes and objects		
4	4	Constructor overloading program		
5	5	Inheritance implementation		
6	6	Method overriding and polymorphism		
7	7	Abstract class and interface implementation		
8	8	Exception handling program		
9	9	Multithreading example		
10	10	Collection framework implementation		

Practical No. 1

Aim: Write a java program for basic arithmetic operations.

Code:

Program 1:

```
public class BasicArithmeticStatic {
   public static void main(String[] args) {
      // Predefined numbers
      int num1 = 25;
      int num2 = 5;

      System.out.println("--- Basic Arithmetic (Static Values) ---");
      System.out.println("Number 1: " + num1);
      System.out.println("Number 2: " + num2);

      // Operations
      System.out.println("Addition (+): " + (num1 + num2));
      System.out.println("Subtraction (-): " + (num1 - num2));
      System.out.println("Multiplication (*): " + (num1 * num2));
      System.out.println("Division (/): " + (num1 / num2));
      System.out.println("Modulus (%): " + (num1 % num2));
   }
}
```

Program 2:

```
import java.util.Scanner;
public class BasicArithmeticInteractive {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      System.out.println("--- Basic Arithmetic (Interactive) ---");

      // Take inputs
      System.out.print("Enter the first number: ");
      double num1 = scanner.nextDouble();

      System.out.print("Enter the second number: ");
      double num2 = scanner.nextDouble();

      // Perform operations and display results
      System.out.println("\n--- Results ---");
      System.out.println("Addition: " + (num1 + num2));
      System.out.println("Subtraction: " + (num1 - num2));
      System.out.println("Multiplication: " + (num1 * num2));

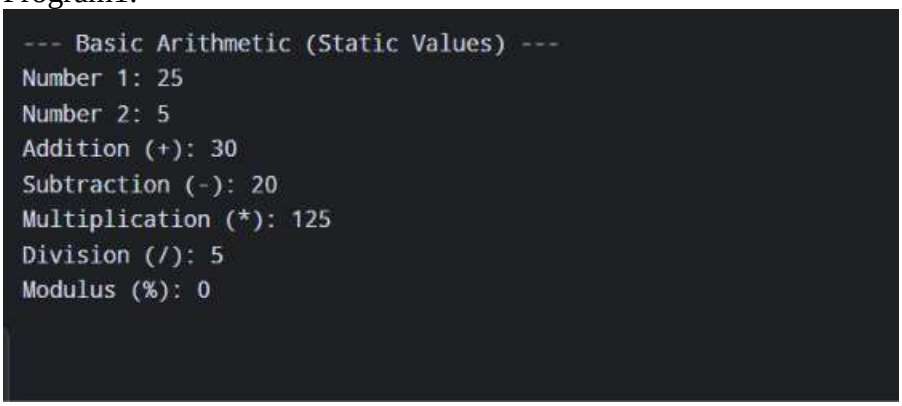
      // Handling division by zero safeguard
```

```
        if (num2 != 0) {
            System.out.println("Division: " + (num1 / num2));
            System.out.println("Modulus: " + (num1 % num2));
        } else {
            System.out.println("Division & Modulus: Cannot divide by zero.");
        }

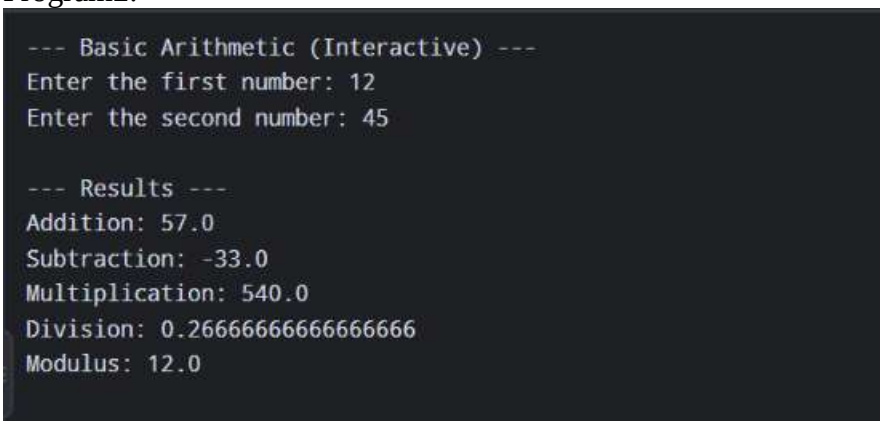
        scanner.close();
    }
}
```

Output:

Program1:



Program2:



Practical No. 2

Aim: Write a java program using conditional and looping statements.

Code:

Program 1:

```
import java.util.Scanner;

public class ConditionalsDemo {
   public static void main(String[] args) {
      Scanner scanner = new Scanner(System.in);

      // Input Year
      System.out.print("Enter a year: ");
      int year = scanner.nextInt();

      // 1. Using IF-ELSE to check for Leap Year
      if ((year % 4 == 0 && year % 100 != 0) || (year % 400 == 0)) {
         System.out.println(year + " is a LEAP YEAR.");
      } else {
         System.out.println(year + " is NOT a leap year.");
      }

      // Input Month Number (1-12)
      System.out.print("Enter month number (1-12): ");
      int month = scanner.nextInt();

      // 2. Using SWITCH statement to determine the season
      String season;
      switch (month) {
         case 12: case 1: case 2:
            season = "Winter";
            break;
         case 3: case 4: case 5:
            season = "Spring";
            break;
         case 6: case 7: case 8:
            season = "Summer";
            break;
         case 9: case 10: case 11:
            season = "Autumn";
            break;
         default:
            season = "Invalid Month!";
            break;
      }

      System.out.println("Season: " + season);
      scanner.close();
   }
}
```

Program 2:

```
import java.util.Scanner;

public class LoopsDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        // 1. FOR LOOP: Print Fibonacci Series
        System.out.print("Enter number of terms for Fibonacci series: ");
        int terms = scanner.nextInt();

        int first = 0, second = 1;
        System.out.print("Fibonacci Series: ");

        for (int i = 1; i <= terms; i++) {
            System.out.print(first + " ");
            int next = first + second;
            first = second;
            second = next;
        }
        System.out.println("\n");

        // 2. WHILE LOOP: Check for Palindrome Number
        System.out.print("Enter a number to check if it's a Palindrome: ");
        int num = scanner.nextInt();

        int originalNum = num;
        int reversedNum = 0;



    }
}
```

Output:

Program1:

```
Enter a year: 2020
2020 is a LEAP YEAR.
Enter month number (1-12): 2
Season: Winter
```

Program2:

```
Enter number of terms for Fibonacci series: 4
Fibonacci Series: 0 1 1 2

Enter a number to check if it's a Palindrome: 123
123 is NOT a palindrome.
```

Practical No. 3

Aim: Write a java program demonstrating classes and objects.

Code:

Program 1:

```
// Class Definition
class Student {
    // Instance variables (Attributes)
    String name;
    int rollNumber;
    double marks;

    // Constructor to initialize object state
    public Student(String name, int rollNumber, double marks) {
        this.name = name;
        this.rollNumber = rollNumber;
        this.marks = marks;
    }

    // Method to display student details
    public void displayDetails() {
        System.out.println("Roll No: " + rollNumber + " | Name: " + name + " | Marks: " + marks);
    }

    // Method to check pass/fail status
    public boolean hasPassed() {
        return marks >= 40.0;
    }
}

public class Main1 {
    public static void main(String[] args) {
        // Creating Object 1
        Student student1 = new Student("Alice", 101, 85.5);

        // Creating Object 2
        Student student2 = new Student("Bob", 102, 32.0);

        // Accessing object methods
        System.out.println("=== Student Records ===");
        student1.displayDetails();
        System.out.println("Passed: " + student1.hasPassed());

        System.out.println();

        student2.displayDetails();
        System.out.println("Passed: " + student2.hasPassed());
    }
}
```

Program 2:

```
import java.util.Scanner;
```

```
// Encapsulated Class
class BankAccount {
    // Private variables (Data Hiding)
    private String accountNumber;
    private double balance;


 // Getter for Balance
    public double getBalance() {
        return balance;
    }

    // Method to Deposit Money
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
            System.out.println("Deposited $" + amount + ". New Balance: $" + balance);
        } else {
            System.out.println("Invalid deposit amount!");
        }
    }

    // Method to Withdraw Money
    public void withdraw(double amount) {
        if (amount > 0 && amount <= balance) {
            balance -= amount;
            System.out.println("Withdrew $" + amount + ". Remaining Balance: $" + balance);
        } else {
            System.out.println("Withdrawal failed! Invalid amount or insufficient balance.");
        }
    }
}
```

Output:

Program1:

```
Initial Balance: $500.0
Deposited $150.0. New Balance: $650.0
Withdrew $200.0. Remaining Balance: $450.0
Withdrawal failed! Invalid amount or insufficient balance.
```

Program2:

```
=== Student Records ===
Roll No: 101 | Name: Alice | Marks: 85.5
Passed: true

Roll No: 102 | Name: Bob | Marks: 32.0
Passed: false
```

Practical No. 4

Aim: Write a java program demonstrating constructor overloading program.

Code:

Program 1:

```
class Rectangle {
    private double length;
    private double width;

    // 1. Default Constructor (No parameters - Unit Square)
    public Rectangle() {
        this.length = 1.0;
        this.width = 1.0;
    }

    // 2. Overloaded Constructor (One parameter - Square)
    public Rectangle(double side) {
        this.length = side;
        this.width = side;
    }

    // 3. Overloaded Constructor (Two parameters - Rectangle)
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    public double calculateArea() {
        return length * width;
    }

    public void displayDetails() {
        System.out.println("Length: " + length + ", Width: " + width + " | Area: " + calculateArea());
    }
}

public class Program1 {
    public static void main(String[] args) {
        System.out.println("=== Rectangle Constructor Overloading ===");

        // Calls Default Constructor
        Rectangle rect1 = new Rectangle();
        rect1.displayDetails();

        // Calls Single-Parameter Constructor
        Rectangle rect2 = new Rectangle(5.0);
        rect2.displayDetails();

        // Calls Two-Parameter Constructor
        Rectangle rect3 = new Rectangle(4.0, 6.0);
        rect3.displayDetails();
    }
```

```
}
```

Program 2:

```
class Book {
    private String title;
    private String author;
    private double price;

    // 1. Constructor with 1 parameter
    public Book(String title) {
        this(title, "Unknown Author", 0.0); // Chains to Constructor 3
    }

    // 2. Constructor with 2 parameters
    public Book(String title, String author) {
        this(title, author, 0.0); // Chains to Constructor 3
    }

    // 3. Fully Parametrized Constructor with 3 parameters
    public Book(String title, String author, double price) {
        this.title = title;
        this.author = author;
        this.price = price;
    }

    public void printSummary() {
        System.out.println("Title: \"" + title + "\" | Author: " + author + " | Price: $" + price);
    }
}

public class Program2 {
    public static void main(String[] args) {
        System.out.println("=== Book Constructor Overloading ===");

        // Creating objects using different constructors
        Book book1 = new Book("Java Basics");
        Book book2 = new Book("Effective Java", "Joshua Bloch");
        Book book3 = new Book("Clean Code", "Robert C. Martin", 45.99);

        book1.printSummary();
        book2.printSummary();
        book3.printSummary();
    }
}
```

Output:

```
=== Book Constructor Overloading ===
Title: "Java Basics" | Author: Unknown Author | Price: $0.0
Title: "Effective Java" | Author: Joshua Bloch | Price: $0.0
Title: "Clean Code" | Author: Robert C. Martin | Price: $45.99
```

```
=== Rectangle Constructor Overloading ===
Length: 1.0, Width: 1.0 | Area: 1.0
Length: 5.0, Width: 5.0 | Area: 25.0
Length: 4.0, Width: 6.0 | Area: 24.0
```

Practical No. 5

Aim: Write a java program demonstrating inheritance implementation.

Code:

Program 1:

```
// Base Class / Parent Class
class Vehicle {
    protected String brand = "Ford";

    public void honk() {
        System.out.println("Beep, beep!");
    }
}

// Derived Class / Child Class (Single Inheritance)
class Car extends Vehicle {
    private String modelName = "Mustang";

    public void displayInfo() {
        // Inherits 'brand' field and 'honk()' method from Vehicle
        System.out.println("Brand: " + brand);
        System.out.println("Model: " + modelName);
    }
}

public class SingleInheritanceDemo {
    public static void main(String[] args) {
        Car myCar = new Car();

        // Call method inherited from Vehicle
        myCar.honk();

        // Call method defined in Car
        myCar.displayInfo();
    }
}}
```

Program 2:

```
// Define the Interface
interface Playable {
    // Interface methods are implicitly 'public' and 'abstract'
    void play();
    void pause();
    void stop();
}

// Class implementing the Interface
class AudioPlayer implements Playable {
    private String songTitle;

    public AudioPlayer(String songTitle) {
        this.songTitle = songTitle;
```

```
    }

    // Must provide public implementations for all interface methods
    @Override
    public void play() {
        System.out.println("Playing audio: " + songTitle);
    }

    @Override
    public void pause() {
        System.out.println("Paused audio: " + songTitle);
    }

    @Override
    public void stop() {
        System.out.println("Stopped playing: " + songTitle);
    }
}

public class BasicInterfaceDemo {
    public static void main(String[] args) {
        // Polymorphic reference: Interface type holding an instance of the implementing class
        Playable player = new AudioPlayer("Bohemian Rhapsody");

        player.play();
        player.pause();
        player.stop();
    }
}
```

Output:

```
Beep, beep!
Brand: Ford
Model: Mustang
```

```
Playing audio: Bohemian Rhapsody
Paused audio: Bohemian Rhapsody
Stopped playing: Bohemian Rhapsody
```

Practical No. 6

Aim: Write a java program demonstrating method overriding and polymorphism.

Code:

Program 1:

```
// Parent Class
class Bank {
    public double getInterestRate() {
        return 0.0; // Base rate
    }
}

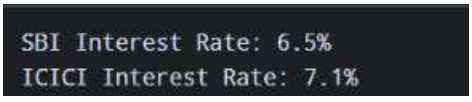
// Child Class 1
class SBI extends Bank {
    @Override
    public double getInterestRate() {
        return 6.5; // SBI specific rate
    }
}

// Child Class 2
class ICICI extends Bank {
    @Override
    public double getInterestRate() {
        return 7.1; // ICICI specific rate
    }
}

public class BankPolymorphismDemo {
    public static void main(String[] args) {
        // Parent class reference pointing to child instances
        Bank bank1 = new SBI();
        Bank bank2 = new ICICI();

        System.out.println("SBI Interest Rate: " + bank1.getInterestRate() + "%");
        System.out.println("ICICI Interest Rate: " + bank2.getInterestRate() + "%");
    }
}
```

Output:



Practical No. 7

Aim: Write a java program demonstrating abstract class and interface implementation.

Code:

Program 1:

```
// Abstract Parent Class
abstract class Animal {
    private String name;

    public Animal(String name) {
        this.name = name;
    }

    // Abstract method (has no body, MUST be implemented by subclasses)
    public abstract void makeSound();

    // Concrete method (has a body, shared by all subclasses)
    public void sleep() {
        System.out.println(name + " is sleeping... Zzz");
    }

    public String getName() {
        return name;
    }
}

// Concrete Subclass 1
class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    // Providing implementation for the abstract method
    @Override
    public void makeSound() {
        System.out.println(getName() + " barks: Woof Woof!");
    }
}

// Concrete Subclass 2
class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

    // Providing implementation for the abstract method
    @Override
    public void makeSound() {
        System.out.println(getName() + " meows: Meow Meow!");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Animal a = new Animal("Generic"); // ERROR! Cannot instantiate an abstract class.

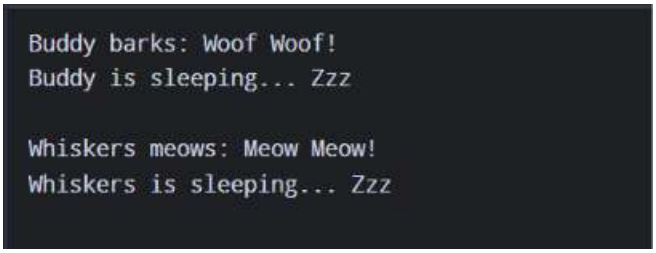
        // Polymorphism: Using abstract class reference for concrete subclass instances
        Animal myDog = new Dog("Buddy");
        Animal myCat = new Cat("Whiskers");

        myDog.makeSound(); // Outputs: Buddy barks: Woof Woof!
        myDog.sleep();     // Outputs: Buddy is sleeping... Zzz

        System.out.println();

        myCat.makeSound(); // Outputs: Whiskers meows: Meow Meow!
        myCat.sleep();     // Outputs: Whiskers is sleeping... Zzz
    }
}
```

Output:



Practical No. 8

Aim: Write a java program demonstrating exception handelling.

Code:

Program 1:

```
public class ExceptionDemo {
    public static void main(String[] args) {
        int numerator = 10;
        int denominator = 0; // Division by zero triggers an ArithmeticException

        try {
            System.out.println("Attempting division...");
            int result = numerator / denominator; // Code that may throw an exception
            System.out.println("Result: " + result); // Skipped if an exception occurs
        }
        catch (ArithmeticException e) {
            // Handles division by zero
            System.out.println("Error: Cannot divide by zero!");
        }
        finally {
            // Always executes, regardless of whether an exception occurred
            System.out.println("Execution completed.");
        }
    }
}
```

Program 2:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.FileNotFoundException;
import java.io.IOException;

public class ReadFileDemo {
    public static void main(String[] args) {
        String filePath = "sample.txt"; // File path to read

        BufferedReader reader = null;

        try {
            // Attempt to open the file
            reader = new BufferedReader(new FileReader(filePath));

            String line;
            System.out.println("Reading file contents:");

            // Read line by line until reaching the end of the file (null)
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
```

```
        } catch (FileNotFoundException e) {
            // Specific catch for when the file does not exist at the given path
            System.out.println("Error: The specified file was not found -> " + filePath);

        } catch (IOException e) {
            // General catch for I/O errors (e.g., read failure, permission denied)
            System.out.println("Error: An I/O error occurred while reading the file.");
            e.printStackTrace();

        } finally {
            // Ensure the reader stream is closed to release system resources
            try {
                if (reader != null) {
                    reader.close();
                    System.out.println("\nFile resource closed successfully.");
                }
            } catch (IOException e) {
                System.out.println("Error closing the reader stream.");
            }
        }
    }
}
```

Output:

```
Attempting division...
ERROR!
Error: Cannot divide by zero!
Execution completed.
```

```
ERROR!
Error: The specified file was not found -> sample.txt
```

Practical No. 9

Aim: Write a java program demonstrating multithreading.

Code:

Program 1:

```
class LoopTask implements Runnable {
    @Override
    public void run() {
        // Loop from 0 to 10 inside each thread
        for (int i = 0; i <= 10; i++) {
            System.out.println(Thread.currentThread().getName() + " - Count: " + i);

            try {
                // Short sleep to allow thread context switching and observe interleaved output
                Thread.sleep(100);
            } catch (InterruptedException e) {
                System.out.println(Thread.currentThread().getName() + " was interrupted.");
            }
        }
    }
}

public class MultiThreadLoopDemo {
    public static void main(String[] args) {
        // 1. Create Runnable task
        LoopTask task = new LoopTask();

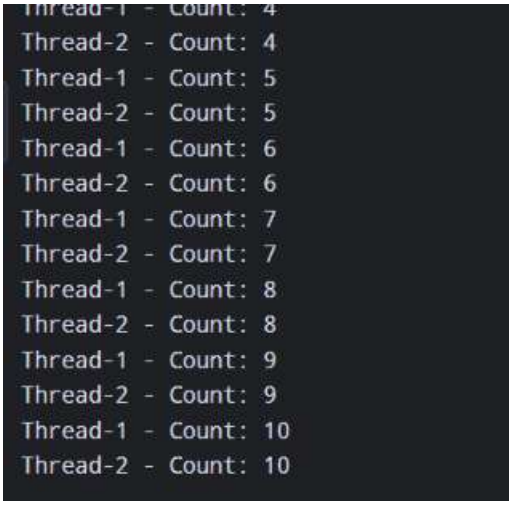
        // 2. Create two threads with custom names
        Thread thread1 = new Thread(task, "Thread-1");
        Thread thread2 = new Thread(task, "Thread-2");

        System.out.println("Starting threads...\n");

        // 3. Start execution of both threads
        thread1.start();
        thread2.start();
    }
}
```

Output:


Starting threads...
Thread-1 - Count: 0
Thread-2 - Count: 0
Thread-1 - Count: 1
Thread-2 - Count: 1
Thread-1 - Count: 2
Thread-2 - Count: 2
Thread-1 - Count: 3
Thread-2 - Count: 3
Thread-1 - Count: 4
Thread-2 - Count: 4
Thread-1 - Count: 5
Thread-2 - Count: 5
Thread-1 - Count: 6
Thread-2 - Count: 6
Thread-1 - Count: 7
Thread-2 - Count: 7
Thread-1 - Count: 8
Thread-2 - Count: 8


Thread-2 - Count: 4
Thread-1 - Count: 5
Thread-2 - Count: 5
Thread-1 - Count: 6
Thread-2 - Count: 6
Thread-1 - Count: 7
Thread-2 - Count: 7
Thread-1 - Count: 8
Thread-2 - Count: 8
Thread-1 - Count: 9
Thread-2 - Count: 9
Thread-1 - Count: 10
Thread-2 - Count: 10

Practical No. 10

Aim: Write a java program demonstrating collection framework.

Code:

Program 1:

```
import java.util.*;

public class CollectionsMasterDemo {

    public static void main(String[] args) {
        System.out.println("========================================");
        System.out.println("   JAVA COLLECTIONS FRAMEWORK MASTER DEMO");
        System.out.println("========================================\n");

        demoArrayList();
        demoLinkedList();
        demoHashSet();
        demoTreeSet();
        demoPriorityQueue();
        demoArrayDeque();
        demoHashMap();
        demoTreeMap();
    }

    // ------------------------------------------------------------
    // 1. ARRAYLIST (Ordered, Allows Duplicates, Fast Index Access)
    // ------------------------------------------------------------
    private static void demoArrayList() {
        System.out.println("--- 1. ArrayList Operations ---");
        List<String> list = new ArrayList<>();

        // Add elements
        list.add("Apple");
        list.add("Banana");
        list.add("Cherry");
        list.add("Banana"); // Duplicates allowed

        // Insert at specific index
        list.add(1, "Blueberry");

        // Access and Modify
        System.out.println("Element at index 2: " + list.get(2));
        list.set(2, "Blackberry"); // Replace "Banana" with "Blackberry"

        // Search
        System.out.println("Contains 'Apple'? " + list.contains("Apple"));
        System.out.println("Index of 'Cherry': " + list.indexOf("Cherry"));

        // Remove
        list.remove("Blackberry"); // Remove by object
        list.remove(0);            // Remove by index
```

```
    // Iteration
    System.out.print("Remaining ArrayList elements: ");
    for (String item : list) {
        System.out.print(item + " ");
    }
    System.out.println("\n");
}

// ---------------------------------------------------------------
// 2. LINKEDLIST (Doubly Linked List, Efficient Insert/Delete)
// ---------------------------------------------------------------
private static void demoLinkedList() {
    System.out.println("--- 2. LinkedList Operations ---");
    LinkedList<Integer> linkedList = new LinkedList<>();

    // Add elements
    linkedList.add(10);
    linkedList.add(20);
    linkedList.add(30);

    // Deque/First-Last specific methods
    linkedList.addFirst(5);  // Add to head
    linkedList.addLast(40);  // Add to tail

    // Peek (Retrieve without removing)
    System.out.println("First Element (getFirst): " + linkedList.getFirst());
    System.out.println("Last Element (getLast): " + linkedList.getLast());

    // Remove operations
    linkedList.removeFirst(); // Removes 5
    linkedList.removeLast();  // Removes 40

    System.out.println("LinkedList contents: " + linkedList + "\n");
}

// ---------------------------------------------------------------
// 3. HASHSET (Unordered, Unique Elements, O(1) Search)
// ---------------------------------------------------------------
private static void demoHashSet() {
    System.out.println("--- 3. HashSet Operations ---");
    Set<String> set = new HashSet<>();

    // Add elements (Duplicates ignored)
    set.add("Red");
    set.add("Green");
    set.add("Blue");
    boolean isAdded = set.add("Red"); // Returns false (duplicate)

    System.out.println("Was second 'Red' added? " + isAdded);
    System.out.println("Set size: " + set.size());

    // Search & Remove
    if (set.contains("Green")) {
        set.remove("Green");
    }
```

```
    System.out.println("HashSet contents (Unordered): " + set + "\n");
}

// ---------------------------------------------------------------
// 4. TREESET (Sorted, Unique Elements, Red-Black Tree)
// ---------------------------------------------------------------
private static void demoTreeSet() {
    System.out.println("--- 4. TreeSet Operations ---");
    TreeSet<Integer> treeSet = new TreeSet<>();

    treeSet.add(40);
    treeSet.add(10);
    treeSet.add(30);
    treeSet.add(20);

    // Natural sorted order automatically maintained
    System.out.println("Sorted TreeSet: " + treeSet);

    // NavigableSet methods
    System.out.println("First (Lowest): " + treeSet.first());
    System.out.println("Last (Highest): " + treeSet.last());
    System.out.println("HeadSet (< 30): " + treeSet.headSet(30));
    System.out.println("TailSet (>= 20): " + treeSet.tailSet(20));
    System.out.println();
}

// ---------------------------------------------------------------
// 5. PRIORITYQUEUE (Min-Heap / Custom Order)
// ---------------------------------------------------------------
private static void demoPriorityQueue() {
    System.out.println("--- 5. PriorityQueue Operations ---");
    // Default Min-Heap (smallest element has highest priority)
    Queue<Integer> pq = new PriorityQueue<>();

    pq.offer(50);
    pq.offer(10);
    pq.offer(30);

    // Peek top element without removal
    System.out.println("Top element (peek): " + pq.peek());

    // Poll elements in order of priority
    System.out.print("Polling elements from PriorityQueue: ");
    while (!pq.isEmpty()) {
        System.out.print(pq.poll() + " "); // 10, 30, 50
    }
    System.out.println("\n");
}

// ---------------------------------------------------------------
// 6. ARRAYDEQUE (Double-Ended Queue / Stack replacement)
// ---------------------------------------------------------------
private static void demoArrayDeque() {
    System.out.println("--- 6. ArrayDeque (Used as Stack & Queue) ---");
    ArrayDeque<String> stack = new ArrayDeque<>();
```

```
    // LIFO (Stack Behavior)
    stack.push("Bottom");
    stack.push("Middle");
    stack.push("Top");

    System.out.println("Popped from Stack: " + stack.pop()); // Top
    System.out.println("Current Stack Top (peek): " + stack.peek());
    System.out.println();
}

// ---------------------------------------------------------------
// 7. HASHMAP (Key-Value Pairs, Unordered, Fast Lookup)
// ---------------------------------------------------------------
private static void demoHashMap() {
    System.out.println("--- 7. HashMap Operations ---");
    Map<String, Integer> map = new HashMap<>();

    // Put Key-Value pairs
    map.put("Alice", 85);
    map.put("Bob", 92);
    map.put("Charlie", 78);

    // Update value for existing key
    map.put("Alice", 90);

    // Access value by key
    System.out.println("Alice's score: " + map.get("Alice"));
    System.out.println("Default score if absent: " + map.getOrDefault("David", 0));

    // Key/Value checks
    System.out.println("Contains key 'Bob'? " + map.containsKey("Bob"));

    // Iterate through entries
    System.out.println("Iterating over HashMap Entries:");
    for (Map.Entry<String, Integer> entry : map.entrySet()) {
        System.out.println("  " + entry.getKey() + " -> " + entry.getValue());
    }
    System.out.println();
}

// ---------------------------------------------------------------
// 8. TREEMAP (Sorted Key-Value Pairs, Red-Black Tree)
// ---------------------------------------------------------------
private static void demoTreeMap() {
    System.out.println("--- 8. TreeMap Operations ---");
    TreeMap<Integer, String> treeMap = new TreeMap<>();

    treeMap.put(103, "Database");
    treeMap.put(101, "Data Structures");
    treeMap.put(102, "Algorithms");

    // Keys are kept in natural ascending order
    System.out.println("Sorted TreeMap: " + treeMap);

    // NavigableMap operations
```

```
        System.out.println("Lowest Key: " + treeMap.firstKey());
        System.out.println("Highest Key: " + treeMap.lastKey());
        System.out.println("SubMap (101 to 103): " + treeMap.subMap(101, 103));
        System.out.println();
    }
}
```

Output:

```
========================================
   JAVA COLLECTIONS FRAMEWORK MASTER DEMO
========================================

--- 1. ArrayList Operations ---
Element at index 2: Banana
Contains 'Apple'? true
Index of 'Cherry': 3
Remaining ArrayList elements: Blueberry Cherry Banana

--- 2. LinkedList Operations ---
First Element (getFirst): 5
Last Element (getLast): 40
LinkedList contents: [10, 20, 30]

--- 3. HashSet Operations ---
Was second 'Red' added? false
Set size: 3
HashSet contents (Unordered): [Red, Blue]
```

```
--- 4. TreeSet Operations ---
Sorted TreeSet: [10, 20, 30, 40]
First (Lowest): 10
Last (Highest): 40
HeadSet (< 30): [10, 20]
TailSet (>= 20): [20, 30, 40]

--- 5. PriorityQueue Operations ---
Top element (peek): 10
Polling elements from PriorityQueue: 10 30 50

--- 6. ArrayDeque (Used as Stack & Queue) ---
Popped from Stack: Top
Current Stack Top (peek): Middle

--- 7. HashMap Operations ---
Alice's score: 90
Default score if absent: 0
Contains key 'Bob'? true
Iterating over HashMap Entries:
  Bob -> 92
```